

Compiladores

Análise Sintática

Ciência da Computação | 6ª etapa

Prof. Dr. Leandro Carlos Fernandes



Análise Sintática DESCENDENTE

```
private $password;  
private $database;  
private $charset;  
  
static public $link = null;  
  
{  
    self::$link = mysql_connect()  
    if (!self::$link)  
        throw new MySQLException("Cannot connect to  
        MySQL. Check the host and password.  
        MySQL Error: " . mysql_error());  
}
```

Métodos Descendentes (Top Down):

Constroem a árvore sintática de cima para baixo (da raiz para as folhas), ou seja, do símbolo inicial da gramática para a sentença.

Analísadores Descendentes Recursivos (ASDR)

Analísadores LL(k)



A análise sintática ...

Tarefa: Dada uma gramática livre de contexto G e uma sentença s , o analisador sintático deve verificar se s pertence a linguagem gerada por G .

O analisador tenta construir a árvore de derivação para s segundo as regras de produção dadas pela gramática G .

Se esta tarefa for possível, o programa é considerado sintaticamente correto.



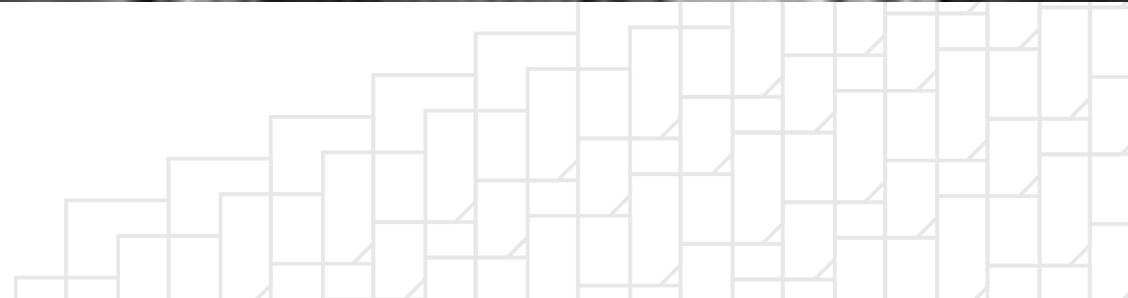
A análise sintática ...

Observe que o analisador não precisa efetivamente construir a árvore, mas sim comprovar que é possível construí-la.

Esse processo pode ser emulado utilizando-se uma pilha de dados.



Analísadores Descendentes Recursivos



Análise Descendente Recursiva (ASDR)

A técnica chamada de descida recursiva (*recursive descent*), é uma forma de análise descendente em que a gramática é transcrita na forma de rotinas responsáveis por processar sua derivação.

Cada símbolo não-terminal se transforma em um procedimento.

Uma chamada ao procedimento A tem a finalidade de encontrar a maior cadeia que pode ser derivada do não-terminal A , a partir do ponto inicial de análise.



Quando realizamos a expansão pela regra $A \rightarrow x_1 x_2 \dots x_n$, o procedimento A faz uma série de chamadas correspondentes aos elementos $x_1, x_2, \dots, x_n$:

Se x_i é um não-terminal, o procedimento x_i é chamado;

Se x_i é um terminal, fazemos uma chamada $check(x_i)$ que verifica a presença do terminal x_i na posição corrente da entrada e aciona o analisador léxico para a obtenção do próximo símbolo.



Suponha a gramática:

$$G = (\{L, S, I\}, \{ (, , ,) , a \}, P, S)$$
$$P = \{ L \rightarrow (S) \}$$
$$S \rightarrow I, S \mid I$$
$$I \rightarrow a \mid L \}$$

Nossa gramática apresenta três regras, então teremos três procedimentos, cada um responsável por verificar o produto derivado a partir cada símbolo não-terminal.

Veja como ficaria ...



Suponha a gramática:

$G = (\{L, S, I\}, \{ (, , ,), a \}, P, S)$

$P = \{ L \rightarrow (S) \}$

$S \rightarrow I, S \mid I$

$I \rightarrow a \mid L \}$

```
void parseL() {  
    check("(");  
    parseS();  
    check(")");  
}
```

Suponha a gramática:

$G = (\{L, S, I\}, \{ (, , ,), a \}, P, S)$

$P = \{ L \rightarrow (S) \}$

$S \rightarrow I, S \mid I$

$I \rightarrow a \mid L \}$

```
void parseS() {  
    parseI();  
    while (tk==",") {  
        check(",");  
        parseI();  
    }  
}
```



Suponha a gramática:

$G = (\{L, S, I\}, \{ (, , ,), a \}, P, S)$

$P = \{ L \rightarrow (S) \}$

$S \rightarrow I, S \mid I$

$I \rightarrow a \mid L \}$

```
void parseI() {  
    switch (tk) {  
        case "a":  
            check("a");  
            break;  
        case "(":  
            parseL();  
            break;  
        default: ERRO();  
    }  
}
```



Análise Descendente Recursiva

- Se a gramática considerada é LL(1), a construção dos procedimentos pode ser feita de maneira automática.
- Por outro lado, se a gramática não é LL(1), ainda é possível construir um analisador de descida recursiva, embora deixa de ser automática.
 - Na prática, este é o caso mais interessante, uma vez que gramática é LL(1), o próximo analisador é sempre mais eficiente do que o de descida recursiva.



Universidade Presbiteriana
Mackenzie



Faculdade de
Computação e Informática

